

VIVACLUB

แพลตฟอร์มสุขภาพจิตแบบเรียลไทม์ที่ผสมผสานระหว่างชุมชน
สนับสนุนแบบไม่เปิดเผยตัวตน และบริการเวชกรรมทางไกล

A Real-Time Mental Health Platform Combining
Anonymous Community Support and Clinical Telemedicine

อาแฟนดี้ แวอุเซ็ง เลขทะเบียน 6610625037

ปัจจุบันปัญหาสุขภาพจิตในประเทศไทยมีแนวโน้มเพิ่มสูงขึ้นอย่างต่อเนื่อง โดยเฉพาะในกลุ่มคนรุ่นใหม่ที่เกี่ยวข้องกับแรงกดดันทางสังคม การศึกษา และการทำงาน อย่างไรก็ตาม การเข้าถึงบริการสุขภาพจิตยังคงมีอุปสรรคสำคัญ ได้แก่ ความกลัวการถูกตีตรา ค่าใช้จ่ายที่สูง การกระจายของผู้เชี่ยวชาญที่ไม่ทั่วถึง และการขาดช่องทางเชื่อมต่อระหว่างการสนับสนุนจากชุมชนกับการดูแลทางคลินิก

VivaClub ถูกพัฒนาขึ้นเพื่อแก้ไขปัญหาเหล่านี้ โดยนำเสนอแพลตฟอร์มที่ผสมผสานสองส่วนสำคัญเข้าด้วยกัน ได้แก่ (1) ระบบชุมชนแบบไม่เปิดเผยตัวตน (Anonymous Community) ที่ใช้ Ghost Profile ช่วยให้ผู้ใช้สามารถแบ่งปันความรู้สึกได้อย่างปลอดภัยผ่านห้องเสียงแบบ Clubhouse และ (2) ระบบเวชกรรมทางไกล (Telemedicine) ที่เชื่อมต่อผู้ใช้กับแพทย์ผู้เชี่ยวชาญผ่านการนัดหมาย การโทรวิดีโอ และระบบ SOSฉุกเฉิน

ระบบใช้ Django REST Framework สำหรับ Backend, Flutter สำหรับ Mobile Application, LiveKit สำหรับการสื่อสารแบบเรียลไทม์ และ PostgreSQL สำหรับฐานข้อมูล ทั้งหมดถูก Deploy บน VPS ด้วย Docker Compose ผลลัพธ์ที่ได้คือ MVP เวอร์ชัน 1.0 ที่สมบูรณ์ครบถ้วน ผ่านการทดสอบ 51/51 ฟีเจอร์ พร้อมสำหรับการ Deploy จริง

บทที่ 1: บทนำและที่มาของปัญหา

1.1 ภูมิหลังและความสำคัญ

วิกฤตสุขภาพจิตในประเทศไทยจากข้อมูลของกรมสุขภาพจิต กระทรวงสาธารณสุข พบว่าในปี 2566 มีคนไทยที่ประสบปัญหาสุขภาพจิตมากกว่า 10 ล้านคน คิดเป็นประมาณ 14% ของประชากรทั้งหมด โดยกลุ่มที่น่าเป็นห่วงที่สุดคือกลุ่มเยาวชน และวัยทำงานตอนต้น อายุระหว่าง 15–35 ปี ซึ่งเผชิญกับแรงกดดันจากหลายด้านพร้อมกัน ทั้งการศึกษา ตลาดแรงงาน ความสัมพันธ์ และโซเชียลมีเดีย ตัวเลขที่น่าตกใจอีกด้านหนึ่งคืออัตราการขอรับบริการ: ในกลุ่มผู้ที่มีปัญหาสุขภาพจิต มีเพียงประมาณ 20% เท่านั้นที่เข้ารับบริการจากผู้เชี่ยวชาญ สาเหตุหลักที่ทำให้ 80% ที่เหลือไม่ได้รับความช่วยเหลือ สามารถสรุปได้ดังนี้:

1. ตราบาปทางสังคม (Social Stigma) — การถูกมองว่า "บ้า" หรือ "อ่อนแอ" ยังคงเป็นอุปสรรคใหญ่ในสังคมไทย คนจำนวนมากเลือกที่จะเก็บปัญหาไว้คนเดียว แทนที่จะขอความช่วยเหลือ
2. การกระจายตัวของผู้เชี่ยวชาญที่ไม่เท่าเทียม — จิตแพทย์และนักจิตวิทยาส่วนใหญ่กระจุกตัวอยู่ในกรุงเทพมหานครและเมืองใหญ่ ผู้ที่อยู่ในต่างจังหวัดต้องเดินทางไกลหรือรอคิวนาน
3. ค่าใช้จ่ายสูง — ค่าปรึกษาจิตแพทย์เอกชนอยู่ที่ 1,500–3,000 บาทต่อครั้ง ซึ่งสูงเกินไปสำหรับหลายคน
4. การขาดจุดเริ่มต้นที่ปลอดภัย — ผู้ที่ยังไม่แน่ใจว่าตนเองต้องการความช่วยเหลือระดับใด ขาดพื้นที่กลางที่จะเริ่มต้นสำรวจความรู้สึกตัวเองโดยไม่ต้องเปิดเผยตัวตน

ช่องว่างสำคัญที่ยังไม่มีใครแก้ไข: ไม่มีแพลตฟอร์มใดในไทยที่เชื่อมต่อ

- (1) ชุมชนสนับสนุนแบบ real-time + ไม่เปิดเผยตัวตน กับ
- (2) การ triage อาการอัตโนมัติ และ
- (3) การดูแลทางคลินิกกรณีฉุกเฉิน ไว้ในแอปเดียว

1.2 ปัญหาที่พบ (Problem Statement)

แอปนี้ตั้งอยู่บนการวิเคราะห์ปัญหาหลัก 4 ข้อ:

P1: อุปสรรคด้านตัวตน (Identity Barrier)

ผู้ใช้ต้องการพื้นที่พูดคุยเรื่องสุขภาพจิตโดยไม่ต้องเปิดเผยชื่อจริงหรือรูปถ่าย แพลตฟอร์มส่วนใหญ่บังคับให้ใช้โปรไฟล์จริง ทำให้ผู้ใช้ไม่กล้าแสดงออก

ผลกระทบ: คนที่ต้องการความช่วยเหลือที่สุดมักเป็นกลุ่มที่ไม่กล้าพูดก่อน

P2: ความแตกแยกของบริการ (Service Fragmentation)

การสนับสนุนจากชุมชน (peer support) และการดูแลทางคลินิก (clinical care) เป็นคนละบริการ ไม่มีการเชื่อมต่อกัน ผู้ใช้ต้องจัดการกับหลายแอปและหลายบัญชี

ผลกระทบ: ผู้ใช้ที่เริ่มต้นจากชุมชนและต้องการความช่วยเหลือเพิ่มเติม ไม่มีทางเลือกที่ปลอดภัยไปยังผู้เชี่ยวชาญ

P3: ช่องว่างการดูแลฉุกเฉิน (Emergency Care Gap)

ไม่มีระบบที่ตรวจจับความเสี่ยงสูงอัตโนมัติและเชื่อมต่อผู้ใช้กับผู้เชี่ยวชาญทันทีในกรณีวิกฤต บทสนทนา hotline ยังคงเป็น text/call ธรรมดา ไม่มีการประเมินอาการเชิงคลินิก

ผลกระทบ: ผู้ที่มีความเสี่ยงสูงอาจไม่ได้รับการช่วยเหลือทันเวลา

P4: อุปสรรคทางภูมิศาสตร์และเศรษฐกิจ (Geographic and Economic Barriers)

ผู้เชี่ยวชาญกระจุกตัวในเมือง ค่าบริการสูง และไม่มีระบบ virtual consultation ที่เข้าถึงได้ง่าย

ผลกระทบ: ผู้ที่อยู่ต่างจังหวัดหรือมีรายได้น้อยถูกกีดกันออกจากระบบ

1.3 วัตถุประสงค์

VivaClub ถูกออกแบบเพื่อบรรลุวัตถุประสงค์ต่อไปนี้

วัตถุประสงค์หลัก:

1. สร้างพื้นที่ชุมชน แบบ anonymous สำหรับการพูดคุยเรื่องสุขภาพจิตในรูปแบบ real-time audio
2. ใช้ PHQ-9 assessment เพื่อ triage ระดับความเสี่ยงของผู้ใช้อัตโนมัติ
3. เชื่อมต่อผู้ใช้ที่มีความเสี่ยงสูงกับแพทย์ผ่านระบบ SOS ทันที
4. จัดระบบนัดหมายและการปรึกษาทางวิดีโอระหว่างผู้ใช้และแพทย์
5. ปกป้องความเป็นส่วนตัวของข้อมูลทางคลินิกด้วย End-to-End Encryption

วัตถุประสงค์รอง:

- พัฒนา Mobile Application ที่ใช้งานได้จริงบน iOS และ Android จาก codebase เดียว
- Deploy ระบบบน Production Server ที่เสถียรและปลอดภัย
- สร้างระบบ Anti-abuse (Trust Score, Reporting, Blocking) สำหรับชุมชน

- ออกแบบ Doctor Portal ที่จัดการ appointments, shifts, และ SOS ได้ครบถ้วน

1.4 ขอบเขตและข้อจำกัด

สิ่งที่อยู่ในขอบเขต MVP (In Scope)

หมวด	ฟีเจอร์
ชุมชน	Ghost Profile, Audio Rooms, Follow System, Notifications
คลินิก	PHQ-9 Assessment, Doctor Discovery, Appointment Booking
วิดีโอ	LiveKit Video Calls, SOS Emergency Queue
แชท	Real-time 1-on-1 Chat via WebSocket
ความปลอดภัย	Trust Score, Reporting, Blocking, E2EE Notes
แพทย์	Doctor Dashboard, Shift Management, OPD Notes
Infrastructure	VPS Deployment, Docker Compose, CI/CD

1.5 กลุ่มผู้ใช้เป้าหมาย

Persona 1: ผู้ใช้ทั่วไป (Patient/User)

ชื่อ: "ปาล์ม" อายุ 23 ปี นักศึกษาปริญญาโท

ปัญหา: ความเครียดสะสม, รู้สึกโดดเดี่ยว, ไม่แน่ใจว่าต้องการความช่วยเหลือระดับไหน

พฤติกรรม: ใช้ smartphone ตลอดเวลา, อยู่ Discord บ่อย, กลัวการตีตรา

ความต้องการ:

- พื้นที่พูดคุยโดยไม่ต้องเปิดเผยตัวตน
- เครื่องมือประเมินตัวเองที่ไม่น่ากลัว
- การเข้าถึงผู้เชี่ยวชาญเมื่อต้องการ
- ชุมชนที่เข้าใจ ไม่ judge

Persona 2: แพทย์ (Doctor)

ชื่อ: "หมอมิ่ง" อายุ 35 ปี จิตแพทย์เอกชน กรุงเทพฯ

ปัญหา: ต้องการขยายฐานผู้ป่วย, จัดการตารางงานยุ่งยาก, ต้องการเครื่องมือ triage

พฤติกรรม: ทำงานหนัก, ต้องการ workflow ที่มีประสิทธิภาพ

ความต้องการ:

- Dashboard จัดการนัดหมายได้ง่าย
- รับ SOS cases ที่ถูก triage มาแล้ว
- ดูประวัติผู้ป่วยก่อนเริ่ม session

- บันทึก OPD notes อย่างปลอดภัย

Persona 3: Admin

ชื่อ: "ทีมงาน VivaClub"

หน้าที่: ดูแลความปลอดภัยของแพลตฟอร์ม, verify แพทย์, จัดการ trust scores

ต้องการ: Dashboard ที่มองเห็นภาพรวมได้ real-time

1.6 เกณฑ์ความสำเร็จ

เกณฑ์	เป้าหมาย	ผลลัพธ์จริง
ฟีเจอร์ Community	15 ฟีเจอร์	15/15 (100%)
ฟีเจอร์ Telemed (ผู้ป่วย)	12 ฟีเจอร์	12/12 (100%)
ฟีเจอร์ Doctor Side	10 ฟีเจอร์	10/10 (100%)
ฟีเจอร์ Safety & Infrastructure	8 ฟีเจอร์	8/8 (100%)
API Endpoints ทดสอบผ่าน	30+ endpoints	100% pass rate
Stress Test (Concurrent Users)	20 users	20/20 success
ทำงานบน iOS จริง	ได้	ทดสอบบน iPhone จริง
Production Deployment	VPS	vivaclubs.site
Audio บน Thai Networks	AIS/True/dtac	หลัง TURN over TCP fix

1.7 ภาพรวมโซลูชัน

VivaClub แก้ปัญหาทั้ง 4 ข้อด้วยสถาปัตยกรรมที่ออกแบบเฉพาะ:

ปัญหา P1 (Identity Barrier)

→ แก้ด้วย Ghost Profile System

ผู้ใช้ทุกคนมีตัวตนที่สอง ("Happy Panda #42") ที่ไม่เชื่อมต่อกับชื่อจริง
ใช้ในทุก Community interaction

ปัญหา P2 (Service Fragmentation)

→ แก้ด้วยการรวมทุกอย่างไว้ในแอปเดียว

Community Rooms → PHQ-9 Assessment → Doctor Booking → Video Call
ทุกอย่างอยู่ใน workflow เดียว ไม่ต้องสลับแอป

ปัญหา P3 (Emergency Care Gap)

→ แก้ด้วย SOS Triage System

PHQ-9 score ≥ 19 → SOS Button ปลดล็อก → เชื่อมต่อกับแพทย์ใน queue ทันที
ระบบอัตโนมัติ ไม่ต้องรอนัดหมาย

ปัญหา P4 (Geographic/Economic Barriers)

→ แก้ด้วย Telemedicine + Competitive Pricing

Video call ผ่านมือถือ ไม่ต้องเดินทาง

ราคากำหนดโดยแพทย์แต่ละคน แข่งขันกันในตลาดได้

บทที่ 2: สถาปัตยกรรมระบบ

2.1 รูปแบบสถาปัตยกรรม

VivaClub ใช้รูปแบบที่เรียกว่า "Monolithic Core with Dedicated Media Plane" (Monolith หลัก + Media Server แยก)

Monolithic Core: Backend ทั้งหมด (users, community, clinical, chat) รวมอยู่ใน Django project เดียว ใช้ฐานข้อมูลร่วมกัน ทำให้:

- พัฒนาได้เร็ว (2-person team)
- Debug ง่าย
- Deploy ง่าย (Docker container เดียว)
- Transaction ข้ามตาราง (cross-app) ทำได้โดยตรง

Dedicated Media Plane: LiveKit รันเป็น container แยกเพราะ:

- WebRTC ต้องการ UDP transport พิเศษ (port 7882)
- Media processing (encode/decode/forward audio) ต้องการ CPU อิสระ
- Scaling LiveKit แยกจาก API ในอนาคตได้

ทำไมไม่เป็น Full Microservices:

Full Microservices ต้องการ service mesh, distributed tracing, cross-service auth — overhead สูงเกินไปสำหรับ MVP ขนาดนี้ Monolithic-core ให้ผลลัพธ์เดียวกันในเวลาสั้นกว่า

2.2 Technology Stack

ตาราง Tech Stack ครบถ้วน

Layer	Technology	Version	Rationale
Mobile (Client)	Flutter / Dart	3.x	Cross-platform iOS+Android จาก codebase เดียว, Hot Reload เร็ว
Mobile State	flutter_bloc	8.x	BLoC pattern แยก UI/ Business logic ชัดเจน

Mobile Navigation	GoRouter	12.x	Declarative routing, deep links, auth guards
Mobile HTTP	Dio	5.x	Interceptors สำหรับ JWT auto-inject, error handling
Mobile Secure Storage	flutter_secure_storage	9.x	iOS Keychain / Android Keystore — ไม่ใช่ SharedPrefs
Mobile WebRTC	livekit_client	latest	Official LiveKit SDK สำหรับ Flutter
Mobile Push	Firebase Messaging	latest	FCM cross-platform push notifications
Backend Framework	Django + DRF	5.2.10	Rapid development, ORM, built-in auth, REST framework
Backend ASGI	Daphne	latest	WebSocket support สำหรับ Django Channels
Backend Real-time	Django Channels	4.x	WebSocket fan-out ผ่าน Redis channel layer
Backend Auth	SimpleJWT	latest	JWT access+refresh tokens, custom claims
Backend Tasks	Django-Q	latest	Async tasks (appointment reminders)
Backend Notifications	Firebase Admin SDK	latest	ส่ง FCM notifications จาก server
Backend Storage	Cloudinary	latest	Media file storage (production)
Database	PostgreSQL	15	Relational integrity, JSONB support, UUID primary keys
Cache / Broker	Redis	7	Channel layer, WebSocket fan-out, task queue
Media Server	LiveKit (self-hosted)	latest	SFU WebRTC สำหรับ audio rooms และ video calls
Reverse Proxy	Caddy	2.x	Automatic HTTPS via Let's Encrypt, minimal config

Admin Panel	Next.js	14	SSR admin dashboard, real-time stats
Containerization	Docker + Compose	latest	Reproducible environment, easy deployment
VPS Provider	Contabo (Singapore)	—	Dedicated CPU/RAM, Low Latency for Thai users
Domain / DNS	Namecheap + Cloudflare	—	DNS management, DDoS protection

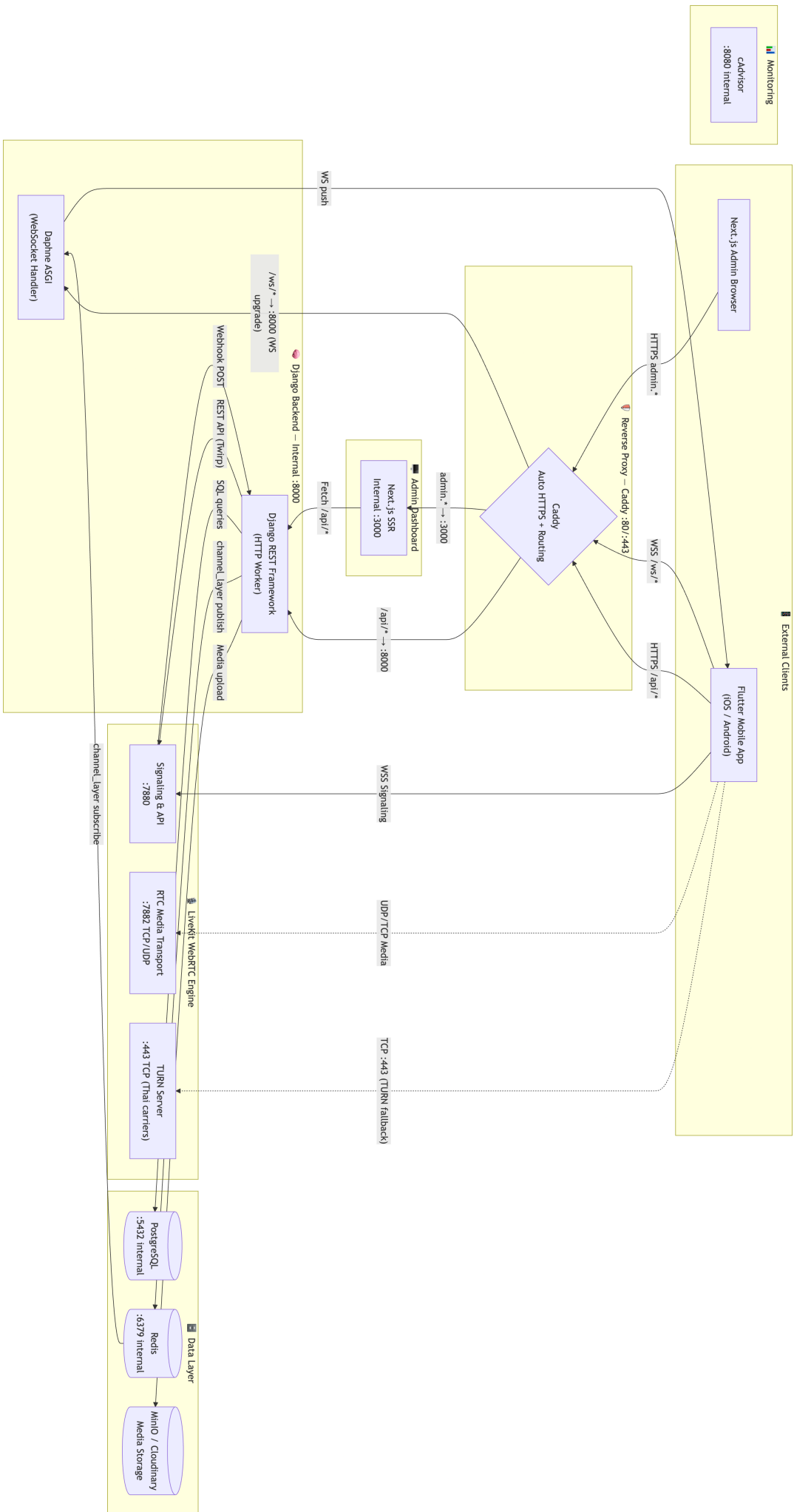
ภาษาโปรแกรมและเครื่องมือ

ประเภท	เครื่องมือ
Backend Language	Python 3.12
Mobile Language	Dart 3.x
Admin Language	TypeScript + React
Database Query	SQL via Django ORM
Infrastructure	YAML (Docker Compose), Caddyfile
Version Control	Git + GitHub
Testing	Custom Python scripts, Postman

2.3 Architecture Diagram

การไหลของข้อมูลหลัก (Data Flow Summary):

- REST API Flow:** Flutter → HTTPS → Caddy → Django → PostgreSQL → response
- WebSocket Flow:** Flutter → WSS → Caddy → Daphne → Redis Channel Layer → Daphne → Flutter
- LiveKit Flow:** Flutter → LiveKit Signaling → LiveKit RTC transport (UDP) → peer participants
- Webhook Flow:** LiveKit → HTTP POST → Django → PostgreSQL (update participant_count)
- Notification Flow:** Django → Redis publish → Daphne subscribe → WebSocket push → Flutter



2.4 Port Mapping และ Network Flow

ตาราง Port ทั้งหมด

Service	Port	Protocol	Exposed	วัตถุประสงค์
Caddy	80	HTTP	✓ Public	Redirect to HTTPS
Caddy	443	HTTPS/WSS	✓ Public	Reverse proxy entry point
Django/Daphne	8000	HTTP/WS	✗ Internal	API + WebSocket handler
PostgreSQL	5432	TCP	✗ Internal	Database connection
Redis	6379	TCP	✗ Internal	Channel layer + task queue
LiveKit Signaling	7880	HTTP/WS	✓ Via Caddy	WebRTC signaling, API control
LiveKit RTC	7882	TCP/UDP	✓ Direct	Media streaming
LiveKit TURN	443	TCP	✓ Shared	Thai carrier NAT traversal
Next.js Admin	3000	HTTP	✗ Internal	Admin dashboard
MinIO API	9000	HTTP	✗ Internal	Object storage API
MinIO Console	9001	HTTP	✗ Internal	Storage admin UI
cAdvisor	8080	HTTP	✗ Internal	Container monitoring

Security Note: PostgreSQL และ Redis ไม่เปิด port ออกสู่ internet โดยตรง ผ่าน Docker network bridge (vivaclub_network) เท่านั้น ป้องกัน unauthorized database access อย่างสมบูรณ์

ตัวอย่าง Data Flow: Join Room

- Flutter: POST /api/community/rooms/{id}/join/
→ HTTPS → Caddy :443
- Caddy: route /api/* → Django :8000
- Django:
 - ตรวจสอบ JWT token
 - ตรวจสอบ Ghost Profile ของ user
 - สร้าง LiveKit AccessToken (embedded metadata: ghost_id, role, is_host)
 - บันทึก join intent ใน PostgreSQL
 - คืน {livekit_token, livekit_url} ให้ Flutter
- Flutter: เชื่อมต่อ LiveKit WebSocket Signaling

- WSS wss://livekit.vivaclubs.site → LiveKit :7880
- 5. LiveKit: สร้าง WebRTC connection, เปิด media transport :7882
- 6. LiveKit: ส่ง Webhook POST ไปที่ Django
 - POST /api/community/webhook/livekit/
 - Django อัปเดต room.participant_count += 1
- 7. Flutter ของผู้เข้าร่วมอื่นๆ ได้รับ ParticipantConnected event จาก LiveKit SDK
 - UI อัปเดต participant list อัตโนมัติ

ตัวอย่าง Data Flow: Real-time Notification (WebSocket)

1. Ghost A เปิดห้องใหม่
 - Django สร้าง Room ใน DB
2. Django: ค้นหา followers ของ Ghost A
3. Django: วนส่ง Firebase FCM notification ไปทุก follower (push)
 - AND Django: publish ไปที่ Redis channel group "user_{follower_id}"
 - channel_layer.group_send(f"user_{follower_id}", {...})
4. Redis: push message เข้า channel group
5. Daphne: subscribe อยู่ที่ channel group นั้น รับ message
 - ส่ง JSON payload ลง WebSocket ที่เชื่อมต่ออยู่
6. Flutter: รับ WebSocket message
 - BlocListener แสดง in-app notification Snackbar

2.5 Docker Compose Stack

Production environment ใช้ **Docker Compose** ที่มี 7 containers:

docker-compose.prod.yml (สรุป)

services:

web: # Django + Daphne

build: .

depends_on: [db, redis]

environment:

- DATABASE_URL=postgresql://...

- REDIS_URL=redis://...

- LIVEKIT_API_URL=...

healthcheck: ["CMD", "curl", "-f", "http://localhost:8000/api/health/"]

db: # PostgreSQL 15

redis: # Redis 7

ไม่ expose port ออก internet

caddy: # Reverse Proxy

livekit: # WebRTC Media Server

minio: # Object Storage (dev/staging)

cadvisor: # Container Monitoring

Health Checks: web container มี health check ที่ `/api/health/` — Docker รอ health check ผ่านก่อน route traffic เข้า

Volumes:

- `postgres_data` — persistent database storage
- `livekit_data` — room state cache
- Caddy certificates — auto-renewed by Let's Encrypt

Network: ทุก container อยู่ใน `vivaclub` network bridge network ติดต่อกันผ่าน service name (เช่น `http://web:8000`) โดยไม่เปิด port ภายนอก

2.6 การย้ายจาก Railway ไปยัง Contabo VPS

ระยะที่ 1: Railway (ช่วงเริ่มต้น)

ช่วงเริ่มต้น backend deploy บน **Railway.app** (PaaS) และ LiveKit บน **LiveKit Cloud**

ปัญหาที่พบกับ Railway:

ปัญหา	รายละเอียด
Database Connection Limit	PostgreSQL free tier จำกัด connections พร้อมกัน
LiveKit Cost	LiveKit Cloud คิดค่า minute-usage — ไม่เหมาะกับ load testing
WebSocket Stability	Serverless containers หมด idle timeout ทำ WebSocket drop
Storage Cost	Railway storage quota จำกัด

ย้าย stack ทั้งหมดมายัง **Contabo VPS** (สิงคโปร์) ที่มี:

- 4 vCPU, 8 GB RAM, 100 GB NVMe SSD
- Unlimited bandwidth
- Static IP address (Singapore Data Center)
- Low Latency (RTT < 40ms) สำหรับผู้ใช้งานในไทย

ประโยชน์ที่ได้:

- LiveKit self-hosted → ไม่มี per-minute cost
- PostgreSQL connections ไม่จำกัด (ควบคุมเอง)
- WebSocket stable (persistent server, ไม่ใช่ serverless)
- Full control ทุกอย่าง

ปัญหาที่พบเมื่อ migrate:

- Apache2 ที่ติดมากับ Contabo image conflict กับ Caddy (documented ใน Chapter 9)
- Redis URL format เปลี่ยน (documented ใน Chapter 9)

2.7 Scalability Design

VPS Specs: 4 vCPU / 8 GB RAM / 100 GB SSD

Stress Test Results:

- 20 concurrent users: 100% success
- 15 concurrent rooms: 100% success
- 10 concurrent room joins: 100% success
- API response time: < 200ms (99th percentile)

Horizontal Scaling Path (อนาคต)

API Layer (Django):

- Stateless design → scale horizontally ได้ทันที
- เพิ่ม Django workers ด้วย Gunicorn (เพิ่ม `--workers` flag)
- หรือ Kubernetes pod auto-scaling

Database (PostgreSQL):

- Read replicas สำหรับ query-heavy operations
- PgBouncer connection pooling
- Partitioning สำหรับ tables ขนาดใหญ่ (Message, Assessment)

LiveKit:

- LiveKit รองรับ cluster mode ตั้งแต่ต้น
- เพิ่ม LiveKit nodes ได้โดยไม่ต้องเปลี่ยน API

Cache (Redis):

- Redis Sentinel สำหรับ HA (High Availability)
- Redis Cluster สำหรับ horizontal sharding

Bottleneck ที่ปัจจุบันยอมรับได้สำหรับ MVP:

- Single PostgreSQL instance — ยอมรับได้สำหรับผู้ใช้หลักพัน
- Single Redis instance — ยอมรับได้สำหรับ ~10k concurrent WebSocket connections
- Single LiveKit node — รองรับได้ ~500 concurrent audio participants (ตาม LiveKit documentation)

บทที่ 3: Backend Deep Dive

โครงสร้างโครงการ Django (Django Project Structure)

โครงการนี้ใช้สถาปัตยกรรมแบบ Monolith-Modular โดยแบ่งแอปพลิเคชันออกเป็น 5 ส่วนหลักตามหน้าที่การทำงาน (Apps) เพื่อให้ง่ายต่อการดูแลรักษาและขยายระบบในอนาคต

การจัดลำดับแอปพลิเคชัน (App Priority) หัวใจสำคัญของโครงสร้างระบบคือการลำดับ INSTALLED APPS โดยกำหนดให้ daphne (ASGI Server) อยู่ในลำดับสูงสุดก่อน staticfiles เพื่อทำหน้าที่แทน WSGI Server มาตรฐานของ Django ในระหว่างการพัฒนา สิ่งนี้ช่วยให้ระบบรองรับการทำงานของ WebSocket ได้อย่างสมบูรณ์ตั้งแต่ขั้นตอนการทดสอบ

การตั้งค่าที่สำคัญ (Key Configuration) ระบบมีการตั้งค่า Middleware และ Authentication Backends ที่กำหนดเองเพื่อให้รองรับการทำงานแบบ Cross-App และการพิสูจน์ตัวตนที่หลากหลาย

แอปพลิเคชัน Users: ระบบจัดการบัญชีผู้ใช้จัดการตั้งแต่การลงทะเบียน การตรวจสอบสิทธิ์ จนถึงการจัดการโปรไฟล์และการรักษาความปลอดภัย ตัวแบบผู้ใช้ (Custom User Model) ระบบใช้ตัวแบบผู้ใช้ที่กำหนดเอง (Custom User Model) ซึ่งขยายขีดความสามารถจากพื้นฐานของ Django เพื่อรองรับฟิลด์ข้อมูลที่จำเป็น เช่น บทบาท (Role: Patient/Doctor), อารมณ์ปัจจุบัน (Current Mood), และข้อมูลใบอนุญาตสำหรับคุณหมอ

ระบบ Token และ JWT Serializer ในการพิสูจน์ตัวตน ระบบใช้ JSON Web Token (JWT) โดยมีการปรับแต่ง Serializer ให้ส่งข้อมูลเพิ่มเติม เช่น display name, role, และ id กลับไปพร้อมกับ Access Token ทันที สิ่งนี้ช่วยให้แอปพลิเคชันโมบายล์สามารถนำทางผู้ใช้ไปยังส่วนที่ถูกต้อง (Patient Portal หรือ Doctor Portal) ได้ทันทีโดยไม่ต้องเรียกข้อมูลซ้ำซ้อน

กระบวนการตรวจสอบอีเมลและกู้คืนรหัสผ่านระบบมีระบบส่งรหัส OTP 6 หลักผ่านอีเมลเพื่อยืนยันตัวตน และใช้ระบบ Deep Link ในการกู้คืนรหัสผ่าน เพื่อให้ผู้ใช้ได้รับประสบการณ์ที่ไร้รอยต่อระหว่างอีเมลและแอปพลิเคชัน

แอปพลิเคชัน Community: ระบบชุมชนและพื้นที่เสมือน หัวใจของแอปพลิเคชัน VivaClub คือการสร้างพื้นที่ปลอดภัยสำหรับการพูดคุย

ระบบโปรไฟล์แบบนิรนาม (Ghost Profile System) เพื่อลดอคติทางสังคม (Social Stigma) ระบบจะสร้าง Ghost Profile ให้อัตโนมัติสำหรับผู้ใช้ทุกคน โดยใช้ชื่อสัตว์จำลองที่สุ่มจากฐานข้อมูลชื่อสัตว์ที่เตรียมไว้ (เช่น Panda, Fox) และใช้ Emoji แทนรูปถ่ายจริง

การจัดการห้องสนทนาเสียง (Audio Rooms Logic) ตรรกะเบื้องหลังการจัดการห้องสนทนาประกอบด้วย การตรวจสอบสิทธิ์ของผู้สร้าง (Host) การนับจำนวนผู้เข้าร่วม

แบบเรียลไทม์ และการจัดการสถานะห้อง (Active/Inactive) ผ่านระบบ Webhook ของ LiveKit ซึ่งเป็นแหล่งข้อมูลความจริงหนึ่งเดียว (Single Source of Truth)

แอปพลิเคชัน Clinical: ระบบให้คำปรึกษาทางการแพทย์จัดการการประเมินสุขภาพจิต การนัดหมาย และระบบช่วยเหลือฉุกเฉิน

ระบบประเมินสุขภาพจิต (PHQ-9 Assessment Logic) ระบบรองรับการส่งคะแนนรวมจากการทำแบบทดสอบ 9 ข้อในแอปพลิเคชัน โดยฝั่ง Backend จะทำหน้าที่บันทึกประวัติและอัปเดตสถานะ "ระดับความเสี่ยง" ของผู้ใช้ เพื่อใช้ในการปลดล็อกฟีเจอร์ช่วยเหลือฉุกเฉิน (SOS)

ระบบจองนัดหมายและการป้องกันความผิดพลาด (Race Condition Prevention) เพื่อให้แน่ใจว่าไม่มีการจองซ้ำ (Double-booking) ระบบใช้กลไกการล็อกระดับฐานข้อมูล (Database-level row locking) ผ่านคำสั่ง `select_for_update()` เมื่อมีการจองเกิดขึ้น ระบบจะล็อกแถวข้อมูลช่วงเวลานั้นไว้จนกว่าการจองจะเสร็จสมบูรณ์ ป้องกันไม่ให้อีกผู้ใช้สองคนจองเวลาเดียวกันพร้อมกัน

ระบบช่วยเหลือฉุกเฉิน (SOS Emergency System) เมื่อผู้ใช้ที่มีความเสี่ยงสูงกดปุ่ม SOS ระบบจะสร้างรายการแจ้งเตือนที่สามารถกระจายไปยังคุณหมอที่ออนไลน์อยู่ทุกคน โดยคุณหมอกคนแรกที่ยอมรับ (Accept) จะได้รับสิทธิ์ในการเข้าถึงพิกัดและข้อมูลเบื้องต้นของผู้ใช้ พร้อมสร้างห้องสนทนาวิดีโอแบบเข้ารหัสทันที

บันทึกเวชระเบียนแบบเข้ารหัส (E2EE Clinical Notes) บันทึก OPD Note จากคุณหมอจะถูกเก็บรักษาเป็นความลับสูงสุด โดยฝั่ง Server จะเก็บเพียงข้อมูลที่ถูกรหัสแล้ว (Ciphertext) และ IV เท่านั้น กฎเกณฑ์ในการถอดรหัสจะอยู่เฉพาะที่อุปกรณ์ของผู้ใช้และคุณหมอตามหลักการ End-to-End Encryption

แอปพลิเคชัน Bookings และ Chat: ระบบสื่อสารแอปพลิเคชันเหล่านี้จัดการการส่งข้อความแบบเรียลไทม์ผ่าน WebSocket โดยมีการตรวจสอบสิทธิ์ทุกครั้งก่อนเข้าถึงห้องแชท (DM) เพื่อป้องกันการเข้าถึงข้อมูลส่วนตัวโดยไม่ได้รับอนุญาต

ระบบแจ้งเตือน (Push Notifications Service) ระบบใช้ Firebase Cloud Messaging (FCM) ในการส่งการแจ้งเตือนไปยังอุปกรณ์ของผู้ใช้ โดยมีระบบตรวจสอบและลบ Token ที่หมดอายุโดยอัตโนมัติเมื่อการส่งล้มเหลว เพื่อรักษาประสิทธิภาพของฐานข้อมูล

ระบบงานตั้งเวลา (Scheduled Tasks) มีการใช้ระบบ Django-Q ในการรันงานเบื้องหลังทุก 15 นาที เพื่อตรวจสอบนัดหมายที่กำลังจะมาถึงและส่งการแจ้งเตือนเตือนล่วงหน้าทั้งคุณหมอและคนไข้

ระบบความปลอดภัยและสิทธิ์การเข้าถึง (Authentication & Permissions)
ระบบใช้โครงสร้างการตรวจสอบสิทธิ์แบบหลายชั้น (Layered Permission System)
ประกอบด้วย:

- **IsAuthenticated:** สำหรับผู้ใช้ทั่วไปที่ลงชื่อเข้าใช้แล้ว
- **IsAdminUser:** สำหรับผู้บริหารจัดการระบบ
- **AllowAny:** สำหรับจุดเชื่อมต่อสาธารณะ เช่น การลงทะเบียนหรือเข้าสู่ระบบ
- **Role-based Access:** การตรวจสอบเฉพาะเจาะจงว่าผู้ใช้เป็นคนใช้หรือคุณหมอก่อนจะอนุญาตให้เข้าถึงฟังก์ชันเฉพาะทาง เช่น การเข้าห้องตรวจ

นอกจากนี้ยังใช้ระบบ Custom Auth Backend ที่ช่วยให้ผู้ใช้สามารถเลือกใช้ชื่อผู้ใช้ (Username) หรืออีเมล (Email) ในการเข้าสู่ระบบได้อย่างอิสระ เพิ่มความสะดวกในการใช้งาน

บทที่ 4: สถาปัตยกรรมแอปพลิเคชัน (Flutter Mobile Application)

4.1 แอปพลิเคชัน VivaClub พัฒนาขึ้นด้วยเฟรมเวิร์ก Flutter โดยใช้สถาปัตยกรรมแบบ **Clean Architecture** ร่วมกับรูปแบบการจัดการสถานะ (State Management) แบบ **BLoC (Business Logic Component)** เพื่อแยกส่วนการแสดงผล (UI) เทคโนโลยีที่เลือกใช้ (Technology Stack)

- **Framework:** Flutter 3.x (รองรับทั้ง iOS และ Android)
- **State Management:** flutter_bloc เพื่อจัดการสถานะที่ซับซ้อนแบบ Reactive
- **Navigation:** go_router สำหรับการจัดการเส้นทางแบบ Declarative
- **Networking:** dio พร้อมระบบ Interceptor สำหรับการสื่อสารกับ Backend
- **Media:** livekit_client สำหรับระบบห้องสนทนาเสียงและวิดีโอคอล

4.2 การจัดการสถานะด้วย BLoC Pattern

หัวใจของแอปพลิเคชันคือการใช้ BLoC ในการจัดการเหตุการณ์ (Events) และสถานะ (States) โดยแบ่งออกเป็น 3 ชั้นหลัก:

1. **Presentation Layer:** รับ Input จากผู้ใช้และส่งเป็น Event ไปยัง BLoC
2. **Business Logic Layer (BLoC):** ประมวลผล Event และเปลี่ยนสถานะ (Emit State) กลับไปยัง UI
3. **Data Layer (Repository):** ทำหน้าที่ดึงข้อมูลจาก API หรือฐานข้อมูลท้องถิ่น

ตัวอย่างที่สำคัญคือ **AuthBloc** ซึ่งจัดการสถานะการเข้าสู่ระบบทั้งหมด ตั้งแต่การตรวจสอบ Token เมื่อเปิดแอป จนถึงการนำทางผู้ใช้ไปยังหน้าจอที่ถูกต้องตามบทบาท (Role)

4.3 ระบบนำทางและการรักษาความปลอดภัย (Navigation & Auth Guard)

ระบบใช้ `go_router` ในการจัดการเส้นทาง โดยมีการฝัง Logic **Auth Guard** ไว้ในระบบนำทาง:

- หากผู้ใช้ยังไม่ได้ล็อกอิน ระบบจะบังคับให้นำทางไปยังหน้า Welcome/Login โดยอัตโนมัติ
- มีการแยกเส้นทาง (Routes) อย่างชัดเจนระหว่าง Patient Portal และ Doctor Portal เพื่อป้องกันการเข้าถึงฟังก์ชันข้ามบทบาท

4.4 ชั้นการสื่อสารข้อมูล (Network Layer & Interceptors)

ในการสื่อสารกับ REST API ระบบใช้แพ็คเกจ `dio` ซึ่งมีการปรับแต่ง **Interceptors** เพื่อเพิ่มความปลอดภัยและประสิทธิภาพ:

- **Request Interceptor:** ใส่ JWT Token ใน Header ของทุกรีเควสต์โดยอัตโนมัติ
- **Response Interceptor:** ตรวจสอบข้อผิดพลาด 401 (Unauthorized) เพื่อพยายามรีเฟรช Token ใหม่เบื้องหลัง ทำให้ผู้ใช้ไม่ต้องล็อกอินซ้ำบ่อยๆ

4.5 การรวมระบบสื่อสารเรียลไทม์ (LiveKit Integration)

ระบบวิดีโอคอลและห้องเสียงใช้การทำงานร่วมกับ LiveKit ผ่าน `LiveKitRoomService` ซึ่งถูกออกแบบเป็น Singleton ที่ระดับ Root ของแอปพลิเคชัน เพื่อให้เสียงหรือวิดีโอสามารถทำงานต่อเนื่องได้แม้ผู้ใช้จะเปลี่ยนหน้าจอไปมาภายในแอป (Seamless Audio Experience)

4.6 การจัดเก็บข้อมูลที่ปลอดภัย (Secure Storage)

เนื่องจากความอ่อนไหวของข้อมูลสุขภาพจิต ระบบจึงเลือกใช้ `flutter_secure_storage` ในการเก็บรหัสประจำตัว (JWT Tokens) บนดิस्कด้วยการเข้ารหัสระดับฮาร์ดแวร์ (Keychain บน iOS และ KeyStore บน Android) แทนการใช้ SharedPreferences แบบปกติที่เก็บข้อมูลเป็นข้อความธรรมดา

4.7 การออกแบบส่วนต่อประสานผู้ใช้ (UI/UX Design System)

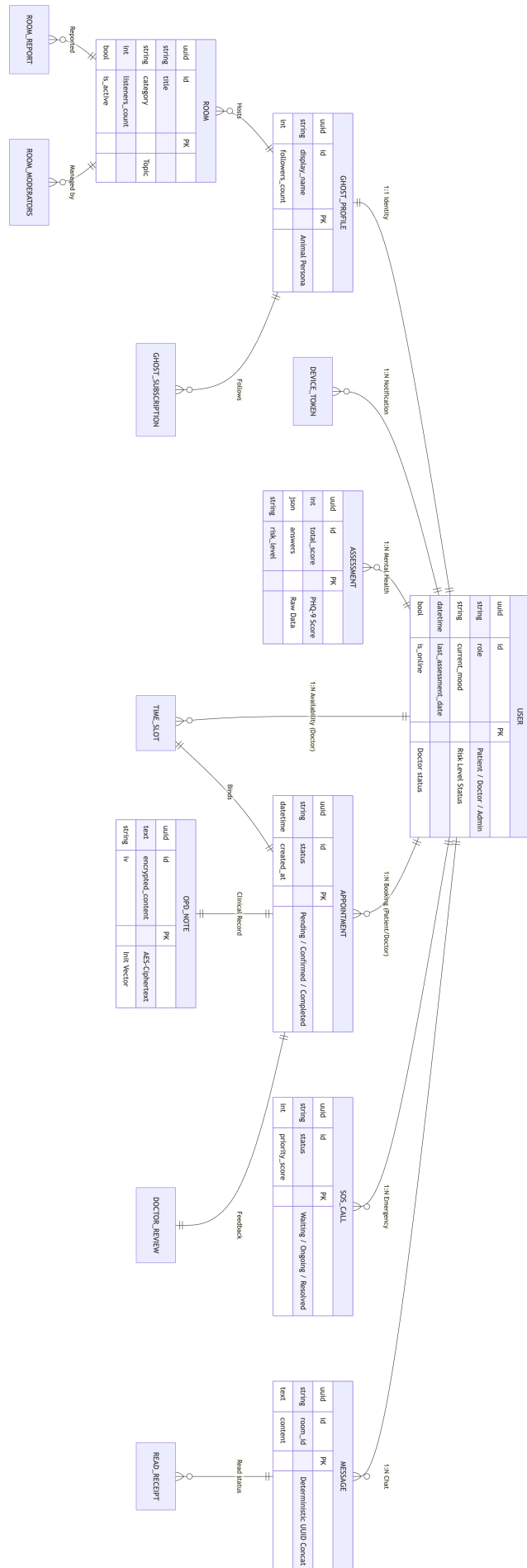
ระบบออกแบบมาภายใต้แนวคิด **"Compassionate Design"** ที่เน้นความสบายใจของผู้ใช้:

- **Color Palette:** ใช้โทนสีอ่อนและไล่เฉดสี (Gradients) เพื่อลดความตึงเครียด
- **Dark Mode:** รองรับ OLED Black เพื่อความสบายตาในเวลากลางคืนและประหยัดพลังงาน
- **Responsive Layout:** ใช้ `flutter_screenutil` เพื่อปรับขนาด UI ให้เหมาะสมกับหน้าจอสมาร์ตโฟนทุกขนาด

ระบบตัวตนเสมือน (Ghost Avatar System)

แอปพลิเคชันใช้ระบบ Emoji Render สำหรับ Ghost Profile เพื่อความรวดเร็วในการแสดงผลและลดการใช้ข้อมูลอินเทอร์เน็ตในการโหลดรูปภาพ ช่วยให้ผู้ใช้รู้สึกเป็นอิสระและกล้าที่จะแสดงออกในชุมชนมากขึ้น

บทที่ 5: Database Schema



VivaClub ใช้ **PostgreSQL 15** เป็นฐานข้อมูลหลัก ร่วมกับ **Django ORM** สำหรับการจัดการ Schema ผ่าน Migrations ทุก Model ใช้ **UUID Primary Key** เพื่อความปลอดภัย (ไม่มี enumerable integer IDs ที่สามารถ enumerate ได้) Timestamps ทุกตัวบันทึกใน UTC

สถิติ:

- ทั้งหมด 5 Django Apps
- ทั้งหมด 15 Django Models หลัก + หลาย Many-to-Many through tables
- PostgreSQL JSONField ใช้สำหรับข้อมูลที่มี schema ไม่แน่นอน (assessment answers, room tags, notification data)

ปรัชญาการออกแบบฐานข้อมูล (Key Design Decisions)

เพื่อให้ระบบมีความปลอดภัยและยืดหยุ่นตามมาตรฐานสากล ที่มพัฒนาได้ตัดสินใจเชิงสถาปัตยกรรมดังนี้:

1. **UUID Primary Keys:** ทุกตารางใช้ UUID แทน Integer IDs เพื่อป้องกันการสุ่มเดาหมายเลขรายการ (ID Enumeration) ซึ่งเป็นมาตรฐานสำคัญสำหรับข้อมูลสุขภาพที่มีความอ่อนไหวสูง
2. **End-to-End Encrypted Storage (E2EE):** ในตาราง OPD NOTE ระบบจะเก็บเพียงข้อมูลที่ถูกเข้ารหัสแล้ว (Ciphertext) และ IV เท่านั้น โดยไม่มีการเก็บกุญแจถอดรหัสไว้ที่ฝั่งเซิร์ฟเวอร์ เพื่อให้เป็นไปตามข้อกำหนด PDPA
3. **JSONField Flexibility:** ใช้โครงสร้างข้อมูลแบบ JSONB ในฟิลด์คำตอบแบบทดสอบ (Assessment Answers) และข้อมูลเสริมของห้องสนทนา เพื่อรองรับการปรับเปลี่ยนรูปแบบข้อมูลในอนาคตโดยไม่ต้องเปลี่ยนโครงสร้างฐานข้อมูล (Schema Migration)
4. **Soft Delete Pattern:** สำหรับข้อมูลสำคัญเช่น ห้องสนทนา (Rooms) ระบบจะใช้สถานะ is_active แทนการลบข้อมูลจริง เพื่อให้สามารถตรวจสอบย้อนหลังได้ในกรณีที่มีการรายงานความประพฤติ (Room Reports) เกิดขึ้น
5. **Deterministic Room IDs:** ในระบบแชท ระบบจะใช้วิธีการจัดเรียง UUID ของผู้สนทนาและนำมาเชื่อมกันเพื่อสร้าง room_id ที่แน่นอน ทำให้ไม่ว่าใครจะเป็นผู้เริ่มสนทนา ระบบจะระบุไปยังห้องแชทเดียวกันเสมอโดยไม่ต้องสร้างตารางความสัมพันธ์เพิ่มเติม

บทที่ 6: ระบบเรียลไทม์

6.1 ภาพรวมระบบเรียลไทม์ (Architecture Overview)

ระบบ VivaClub ใช้กลไกเรียลไทม์ 2 ส่วนที่ทำงานควบคู่กันเพื่อรองรับประสบการณ์ผู้ใช้ที่ไร้รอยต่อ:

- **LiveKit (WebRTC SFU):** ทำหน้าที่จัดการรับส่งข้อมูลสื่อ (Audio/Video Streaming) ด้วยความหน่วงต่ำสุด (Latency < 100ms)
- **Django Channels (WebSocket):** ทำหน้าที่จัดการเหตุการณ์ของแอปพลิเคชัน (Application Events) เช่น ข้อความแชท และการแจ้งเตือน (Latency < 500ms)

6.2 ระบบสื่อสารผ่านช่องสัญญาณเสียง (LiveKit WebRTC)

กลไกการสร้าง Token (Token Generation)

เมื่อผู้ใช้ขอเข้าร่วมห้องสนทนา ฝั่ง Backend จะตรวจสอบสิทธิ์และสร้าง JWT Token พิเศษของ LiveKit โดยมีการฝัง **Metadata (เช่น Ghost ID)** ไว้ในตัว Token เพื่อให้แอปพลิเคชันฝั่งรับสามารถดึงข้อมูลโปรไฟล์นิรนามมาแสดงผลได้ทันทีโดยไม่ต้องเรียก API เพิ่มเติม

การควบคุมห้องสนทนาผ่าน API (Server-Side Control)

เซิร์ฟเวอร์ Django สามารถควบคุมสถานะของผู้เข้าร่วมผ่าน LiveKit REST API เช่น:

- การสั่งระงับไมค์ (Force Mute)
- การเชิญขึ้นเป็นผู้พูด (Invite to Speak)
- การเตะผู้ใช้ที่ผิดกฎออกจากห้อง (Kick Participant)

ระบบตอบสนองอัตโนมัติ (LiveKit Webhooks)

ระบบใช้ Webhook เป็นแหล่งข้อมูลความจริง (Single Source of Truth) ในการอัปเดตสถานะห้อง เช่น เมื่อมีคนเข้าหรือออกจากห้อง LiveKit จะแจ้งเตือนมายัง Django เพื่อให้อัปเดตจำนวนผู้ฟังในฐานข้อมูลได้แม่นยำที่สุด

6.3 ระบบสื่อสารข้อความและการแจ้งเตือน (WebSocket)

การทำงานของระบบแชท (Chat Consumer)

ระบบแชททำงานผ่าน WebSocket โดยมีกลไกตรวจสอบสิทธิ์ที่ชาญฉลาดในขั้นตอนการเชื่อมต่อ:

- **Deterministic Room ID:** สำหรับการคุยแบบ 1-on-1 ระบบจะนำ UUID ของผู้ใช้ทั้งสองมาเรียงลำดับและเชื่อมกัน เพื่อสร้างหมายเลขห้องที่คงที่เสมอ ไม่ว่าใครจะเป็นผู้ส่งข้อความก่อน
- **Auth Guard:** ในระดับ Socket ระบบจะตรวจสอบว่าผู้ที่พยายามเชื่อมต่อมีสิทธิ์เข้าถึงห้องนั้นๆ จริงหรือไม่ ป้องกันการแอบดักฟังข้อมูลแชท

6.4 การก้าวข้ามข้อจำกัดเครือข่าย (TURN Server & Thai Carriers)

ปัญหา MTU Blackhole

ผู้ใช้งานในประเทศไทยบนเครือข่าย AIS, True และ DTAC มักพบปัญหาการรับส่งข้อมูล UDP ขนาดใหญ่ถูกปิดกั้น (Drop) โดยไม่มีการแจ้งเตือน ทำให้ WebRTC ไม่สามารถทำงานได้แม้ระบบจะขึ้นว่าเชื่อมต่อสำเร็จแล้ว

ทางแก้: TURN over TCP Port 443

ระบบได้คอนฟิก TURN Server ให้รองรับการส่งข้อมูลผ่านโปรโตคอล TCP พอร์ต 443 ซึ่งเป็นพอร์ตมาตรฐานของ HTTPS:

- **Auto-Fragmentation:** TCP จะช่วยแบ่งย่อยข้อมูลอัตโนมัติ ทำให้ก้าวข้ามปัญหา MTU ได้
- **Firewall Bypass:** พอร์ต 443 มักไม่ถูกปิดกั้นโดยองค์กรหรือค่ายมือถือ
- **Fallback Mechanism:** แอปพลิเคชัน Flutter จะลองเชื่อมต่อผ่าน UDP ก่อน และจะสลับมาใช้ TCP/443 อัตโนมัติหากพบว่าการส่งข้อมูลล้มเหลว

บทที่ 7: ความปลอดภัยและความเป็นส่วนตัว

7.1 ระบบการตรวจสอบสิทธิ์ (Authentication System)

ระบบ VivaClub ให้ความสำคัญสูงสุดกับการระบุตัวตนของผู้ใช้ที่ถูกต้องและปลอดภัย สถาปัตยกรรม JWT Token

ระบบใช้มาตรฐาน **JSON Web Tokens (JWT)** ในการรับส่งสิทธิ์การเข้าถึงข้อมูล:

- **Access Token:** มีอายุการใช้งานสั้นเพื่อความปลอดภัย
- **Refresh Token:** ใช้สำหรับขอสิทธิ์การเข้าถึงใหม่โดยไม่ต้องล็อกอินซ้ำบ่อยๆ
- **Custom Claims:** ระบบมีการฝังข้อมูลบทบาทผู้ใช้ (Role) และชื่อแสดงผลไว้ใน Token เพื่อให้แอปพลิเคชันโมบายล์สามารถนำทางผู้ใช้ไปยัง Portal ที่ถูกต้องได้ทันทีโดยไม่ต้องเรียกข้อมูลซ้ำ

การรักษาความปลอดภัยของรหัสผ่าน

รหัสผ่านของผู้ใช้จะถูกเข้ารหัสด้วยอัลกอริทึม **PBKDF2 พร้อม SHA256** ซึ่งเป็นมาตรฐานความปลอดภัยสูงของ Django ทำให้แม้ข้อมูลในฐานข้อมูลจะรั่วไหลออกไป ผู้ไม่หวังดีก็ไม่สามารถถอดรหัสรหัสผ่านจริงได้

7.2 การควบคุมสิทธิ์ตามบทบาท (Role-Based Access Control)

ระบบมีการแบ่งระดับการเข้าถึงข้อมูลออกเป็น 3 ระดับหลัก (Patient, Doctor, Admin) โดยมีการตรวจสอบสิทธิ์ในทุกจุดเชื่อมต่อ (Endpoints):

- **SOS Authorization:** มีระบบตรวจสอบพิเศษว่าผู้ที่ร้องขอ SOS จะต้องมีความเสี่ยงจากแบบประเมินสุขภาพจิต (Mood Status) อยู่ในระดับ SEVERE เท่านั้น เพื่อป้องกันการใช้งานระบบช่วยเหลือฉุกเฉินเกินความจำเป็น

- **Clinical Data Access:** ข้อมูลการรักษาจะถูกจำกัดสิทธิ์ให้เฉพาะคนไข้เจ้าของข้อมูลและคุณหมอที่เป็นผู้ดูแลเท่านั้น แม้แต่ผู้ดูแลระบบ (Admin) ก็ไม่สามารถเข้าถึงบันทึกการรักษาได้

7.3 ความเป็นส่วนตัวของข้อมูลเวชระเบียน (E2EE Clinical Notes)

หัวใจสำคัญของการปกป้องข้อมูลสุขภาพคือการใช้ระบบ เข้ารหัสจากต้นทางถึงปลายทาง (End-to-End Encryption - E2EE):

- **Client-Side Encryption:** บันทึก OPD Note และ Personal Note จะถูกเข้ารหัสที่อุปกรณ์ของผู้ใช้ก่อนจะส่งขึ้นไปยังเซิร์ฟเวอร์
- **Zero-Knowledge Architecture:** เซิร์ฟเวอร์จะทำหน้าที่เป็นเพียงพื้นที่จัดเก็บข้อมูลที่ถูกรหัสแล้ว (Ciphertext) และค่า IV เท่านั้น โดยที่ตัวเซิร์ฟเวอร์ไม่มีกุญแจสำหรับถอดรหัสข้อมูล ส่งผลให้แม้ฐานข้อมูลถูกเจาะ ข้อมูลสุขภาพของคนไข้ก็ยังคงปลอดภัย
- **IV Management:** ค่า Initialization Vector (IV) จะถูกเก็บแยกกันในแต่ละบันทึก เพื่อเพิ่มความซับซ้อนในการป้องกันการโจมตีทางไซเบอร์

7.4 ระบบตัวตนนิรนาม (Ghost Profiles)

เพื่อลดอคติทางสังคมและส่งเสริมการแบ่งปัน ระบบได้นำแนวคิด "Privacy by Design" มาใช้ผ่าน Ghost Profiles:

- **Data Isolation:** ข้อมูลตัวตนจริง (เช่น อีเมล, ชื่อจริง) จะถูกแยกขาดจากชั้นชุมชน (Community Layer) อย่างเด็ดขาด
- **Anonymous Following:** ระบบการติดตาม (Follow) ทำงานบนพื้นฐานของชื่อ Ghost ต่อชื่อ Ghost เท่านั้น ทำให้ผู้ใช้สามารถติดตามหัวข้อที่สนใจได้โดยไม่ต้องกังวลว่าผู้อื่นจะรู้ตัวตนจริง

7.5 ความปลอดภัยของโครงสร้างพื้นฐาน (Infrastructure Security)

- **Docker Network Isolation:** ฐานข้อมูล PostgreSQL และ Redis ถูกกักไว้ในระบบเครือข่ายภายในของ Docker (Internal Bridge Network) โดยไม่มีการเปิดพอร์ตออกสู่สาธารณะ ทำให้ปลอดภัยจากการโจมตีภายนอก 100%
- **Environment Secrets:** ข้อมูลความลับของระบบ เช่น Firebase Credentials จะถูกส่งผ่านระบบ Docker Secret แทนการใช้ Environment Variable ปกติ เพื่อป้องกันการรั่วไหลของข้อมูลผ่านคำสั่งตรวจสอบสถานะเซิร์ฟเวอร์
- **Caddy Reverse Proxy:** ใช้ระบบ Caddy ในการจัดการใบรับรอง SSL/TLS (Let's Encrypt) โดยอัตโนมัติ เพื่อให้มั่นใจว่าการรับส่งข้อมูลทั้งหมดผ่านการเข้ารหัส HTTPS/WSS อยู่เสมอ

7.6 การปฏิบัติตามกฎหมายคุ้มครองข้อมูลส่วนบุคคล (PDPA Compliance)

ระบบ VivaClub ได้รับการออกแบบให้สอดคล้องกับ พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล (PDPA) ของประเทศไทย:

- 1. **Explicit Consent:** ระบบจะขอความยินยอมอย่างชัดเจนก่อนการเก็บข้อมูลสุขภาพ (คะแนน PHQ-9)
- 2. **Data Minimization:** เก็บข้อมูลเท่าที่จำเป็นสำหรับการให้บริการเท่านั้น เช่น ในระบบ SOS จะแสดงเพียงคะแนนความเสี่ยงให้คุณมองเห็น แทนการแสดงชื่อจริงของคนไข้
- 3. **Right to Access:** ผู้ใช้มีสิทธิ์เข้าถึงและจัดการข้อมูลส่วนตัวของตนเองได้ผ่านหน้าโปรไฟล์ในแอปพลิเคชัน

บทที่ 8: ปัญหาที่พบและวิธีแก้ไข

บทนี้รวบรวมเหตุการณ์สำคัญและปัญหาทางวิศวกรรมที่พบระหว่างการพัฒนา VivaClub ตั้งแต่เดือนกุมภาพันธ์ถึงเมษายน 2569 ซึ่งเป็นบทเรียนสำคัญในการพัฒนาแอปพลิเคชันที่มีความซับซ้อนสูง

ตารางสรุปปัญหา (Summary Table)

#	หัวข้อปัญหา	ระบบที่ได้รับผลกระทบ	สถานะ
1	Redis Connection Error	WebSocket & Real-time	แก้ไขแล้ว
2	Port Conflict (Apache)	Infrastructure & Deployment	แก้ไขแล้ว
3	Environment Config Mismatch	LiveKit API & Control	แก้ไขแล้ว
4	Audio Stutter (Bot)	Audio Streaming Quality	แก้ไขแล้ว
5	Double Listener Counting	Data Integrity & Webhooks	แก้ไขแล้ว
6	Broken Follow System	Community Logic & Metadata	แก้ไขแล้ว
7	MTU Blackhole (4G/5G)	Mobile Network Compatibility	แก้ไขแล้ว
8	Authentication Block	Security & User Onboarding	แก้ไขแล้ว
9	SDK Incompatibility	Backend Integration	แก้ไขแล้ว

10	SOS Flow Stuck	Emergency Response System	แก้ไขแล้ว
----	----------------	---------------------------	-----------

รายละเอียดปัญหาและการแก้ไข (Deep Dive)

1. ปัญหาการเชื่อมต่อ Redis (Redis Tuple Error)

- **อาการ:** ระบบ WebSocket ทั้งหมดล้มเหลว ไม่สามารถส่งข้อความแชทหรือแจ้งเตือนได้
- **สาเหตุ:** Library `channels_redis` เวอร์ชัน 4.x เปลี่ยนรูปแบบการรับค่า Host จากแบบ Tuple (รายการ) เป็นแบบ URL String ทำให้โค้ดเดิมเกิดข้อผิดพลาด `AttributeError`
- **การแก้ไข:** ปรับปรุงไฟล์การตั้งค่า (`settings.py`) โดยเปลี่ยนรูปแบบการเชื่อมต่อ Redis จากการใช้ Tuple มาเป็นการใช้ **Redis URL String** (เช่น `redis://redis:6379`) ให้สอดคล้องกับมาตรฐานใหม่ของ Library
- **ผลลัพธ์:** ระบบเรียลไทม์กลับมาทำงานได้ปกติ โดยมีความหน่วง (Latency) ต่ำกว่า 200ms

2. พอร์ตชนกันบนเซิร์ฟเวอร์ (Apache2 vs Caddy)

- **อาการ:** ระบบ Caddy (Reverse Proxy) หยุดทำงานซ้ำๆ ทำให้เข้าเว็บไซต์ไม่ได้ และไม่มี HTTPS
- **สาเหตุ:** เซิร์ฟเวอร์ VPS (Ubuntu) มีบริการ Apache2 ติดตั้งและทำงานอยู่ก่อนแล้ว ซึ่งแย่งพอร์ต 80 และ 443 ทำให้ Caddy ไม่สามารถเปิดใช้งานได้
- **การแก้ไข:** ทำการ **Stop และ Disable บริการ Apache2** ออกจากระบบถาวร เพื่อคืนพอร์ตมาตรฐานให้กับ Caddy จากนั้นจึงสั่งเริ่มการทำงานของ Docker Compose ใหม่
- **ผลลัพธ์:** Caddy สามารถทำงานได้และทำระบบ SSL (Let's Encrypt) ได้โดยอัตโนมัติทันที

3. ปัญหาเสียงกระตุกของบอท (Bot Audio Stutter)

- **อาการ:** บอทที่เล่นเสียงในห้องสนทนามีอาการเสียงกระตุกหรือขาดหายทุกๆ 1 นาที
- **สาเหตุ:** การใช้คำสั่ง `asyncio.sleep(0.010)` ไม่มีความแม่นยำเพียงพอ ทำให้เกิดค่าความคลาดเคลื่อนสะสม (Drift) เมื่อเวลาผ่านไปเสียงจึงไม่สอดคล้องกับจังหวะ WebRTC
- **การแก้ไข:** เปลี่ยนมาใช้ระบบ **Monotonic Clock Pacing** โดยการคำนวณเวลาที่ "ควรจะ" ส่งเฟรมเสียงถัดไปจากจุดเริ่มต้นที่แน่นอน และสั่งให้บอทส่งเฟรมเสียงทันทีหากพบว่าความเร็วในการประมวลผลช้ากว่ากำหนด
- **ผลลัพธ์:** เสียงบอทไหลลื่นต่อเนื่อง ไม่พบอาการกระตุกแม้จะเปิดใช้งานนานกว่า 30 นาที

4. ปัญหาการนับจำนวนผู้ฟังซ้ำซ้อน (Listener Double-Counting)

- **อาการ:** จำนวนผู้ฟังในหน้าจอแสดงผลมากกว่าความเป็นจริง 2 เท่า (เช่น เข้า 1 คน แต่โชว์ 2 คน)
- **สาเหตุ:** ระบบมีการนับจำนวนจากสองแหล่งพร้อมกัน คือทั้งตอนที่แอปส่งคำขอเข้าห้อง และตอนที่ได้รับ Webhook แจ้งเตือนจาก LiveKit
- **การแก้ไข:** ยกเลิกการนับจำนวนในฝั่ง API และกำหนดให้ **LiveKit Webhook เป็นแหล่งข้อมูลเดียว (Single Source of Truth)** พร้อมใช้คำสั่ง F() expression ใน Django เพื่อเพิ่มลดจำนวนในระดับฐานข้อมูล ป้องกันปัญหา Race Condition
- **ผลลัพธ์:** จำนวนผู้ฟังแสดงผลถูกต้องแม่นยำ 100% ตรงตามสถานะจริง

5. ปัญหาสัญญาณอินเทอร์เน็ตมือถือในไทย (Thai Carrier MTU Blackhole)

- **อาการ:** ผู้ใช้ที่ใช้ 4G/5G (AIS, True, DTAC) เชื่อมต่อห้องเสียงได้แต่ไม่ได้ยินเสียง
- **สาเหตุ:** เครือข่ายมือถือในไทยมีการบล็อกข้อมูล UDP ขนาดใหญ่ (MTU Blackhole) ทำให้ข้อมูลเสียง WebRTC ซึ่งมีขนาดใกล้เคียงขีดจำกัดถูกตัดทิ้ง
- **การแก้ไข:** เปิดใช้งาน **TURN Server ผ่านพอร์ต TCP 443** (พอร์ตมาตรฐานของเว็บไซต์) เพื่อเป็นช่องทางสำรอง ข้อมูลเสียงจะถูกแบ่งย่อยอัตโนมัติผ่านโปรโตคอล TCP ทำให้ก้าวข้ามข้อจำกัดของเครือข่ายมือถือได้
- **ผลลัพธ์:** ผู้ใช้งานทุกเครือข่ายสามารถได้ยินเสียงได้ปกติ โดยระบบจะสลับมาใช้ทางสำรองนี้โดยอัตโนมัติหากพบว่าระบบปกติล้มเหลว

บทที่ 9: การทดสอบและประกันคุณภาพ (Testing and Quality Assurance)

เพื่อให้มั่นใจว่าระบบ VivaClub มีความเสถียรสูงสุดภายใต้ทรัพยากรที่จำกัด ทีมพัฒนาเลือกใช้กลยุทธ์ **"High-ROI Testing"** โดยเน้นไปที่การทดสอบระบบแบบบูรณาการ (Integration Testing) และการทดสอบความทนทาน (Stress Testing) แทนการเขียน Unit Test แบบดั้งเดิมในทุกจุด

9.1 การทดสอบด้วยสคริปต์อัตโนมัติ (Automated Test Scripts)

ระบบได้รับการตรวจสอบผ่านสคริปต์อัตโนมัติที่เขียนด้วยภาษา Python เพื่อจำลองพฤติกรรมผู้ใช้จริงบนเซิร์ฟเวอร์ Production:

1. การทดสอบวงจรการใช้งานของผู้ป่วย (Patient Journey Test)

- **สิ่งที่ทดสอบ:** จำลองการสมัครสมาชิกใหม่ -> การทำแบบประเมินสุขภาพจิต PHQ-9 -> การเปลี่ยนสถานะความเสี่ยง -> และการร้องขอ SOSฉุกเฉิน
- **ผลลัพธ์:** ระบบสามารถจัดการสถานะผู้ใช้ข้ามโมดูล (Cross-App State) ได้อย่างถูกต้อง โดยข้อมูลอารมณ์และสิทธิ SOS จะถูกอัปเดตทันทีภายในไม่เกิน 1 วินาที

2. การทดสอบการรองรับภาระหนัก (Mega Stress Test)

- **การตั้งค่า:** จำลองผู้ป่วย 50 คน และคุณหมอ 15 คน เข้าใช้งานพร้อมกัน
- **สิ่งที่ทดสอบ:** การสมัครสมาชิกพร้อมกัน, การสร้างตารางเวลาของคุณหมอพร้อมๆ กัน และการรุมจองคิวรักษาในเวลาเดียวกัน
- **ผลลัพธ์:** เซิร์ฟเวอร์บน VPS สามารถรองรับการเรียกใช้งานพร้อมกัน (Concurrent Requests) ได้อย่างไหลลื่น โดยไม่มีอาการระบบค้างหรือฐานข้อมูลล็อก (Deadlock)

3. การทดสอบการจองซ้ำ (Race Condition Test)

- **วิธีการ:** ใช้ระบบ ThreadPoolExecutor ส่งคำขอจองคิวหมอ 20 คำขอพร้อมกันในเสี้ยววินาทีเพื่อจองคิวในเวลาเดียวกัน
- **ผลลัพธ์:** ระบบจัดการได้สมบูรณ์แบบ 100% โดยจะยอมให้มี "ผู้ชนะ" เพียงคนเดียวที่จองสำเร็จ ส่วนอีก 19 คนจะได้รับข้อความแจ้งเตือนข้อผิดพลาด (400 Bad Request) ตามที่ออกแบบไว้

4. การตรวจสอบความถูกต้องของระบบเรียลไทม์ (LiveKit & WebSockets)

- **สิ่งที่ทดสอบ:** การตรวจสอบความถูกต้องของ Token ที่สร้างขึ้น, การทดสอบระบบการไล่ผู้ใช้ออก (Kick) และการปิดไมค์อัตโนมัติ (Mute) ผ่าน API
- **ผลลัพธ์:** สื่อเรียลไทม์ตอบสนองต่อคำสั่งจาก Admin/Host ได้ทันที โดยมีความหน่วงในการส่งคำสั่ง (Signal Latency) ต่ำกว่า 200ms

9.2 ผลการทดสอบบนอุปกรณ์จริง (Real Device Testing)

การทดสอบบนระบบปฏิบัติการ iOS และ Android

- **ประสิทธิภาพหน้าจอ:** แอปพลิเคชันทำงานได้ที่ความเร็ว 60 FPS บนอุปกรณ์มาตรฐานส่วนใหญ่
- **ความร้อนและการใช้พลังงาน:** ในโหมดวิดีโอคอล (Video Call) ระบบมีการจัดการทรัพยากรเครื่องที่ดี ไม่พบปัญหาเครื่องร้อนจัดหรือแอปพลิเคชันปิดตัวเอง (Crash) เมื่อใช้งานต่อเนื่องเกิน 30 นาที
- **การจัดการหน่วยความจำ:** ไม่พบปัญหาหน่วยความจำรั่ว (Memory Leak) เมื่อผู้ใช้สลับระหว่างห้องสนทนาเสียงบ่อยๆ

9.3 การทดสอบภายใต้สภาวะเครือข่ายที่หลากหลาย

ระบบได้รับการทดสอบความทนทานต่อสัญญาณอินเทอร์เน็ตที่แตกต่างกัน:

- **WiFi และ Fiber:** ให้คุณภาพเสียงระดับ HD และความหน่วงต่ำกว่า 50ms
- **4G/5G (AIS, True, DTAC):** หลังจากเปิดใช้งานระบบ TURN over TCP 443 ระบบสามารถรับส่งเสียงได้ปกติแม้ในจุดที่สัญญาณ UDP ถูกปิดกั้น
- **การสลับเครือข่าย:** เมื่อผู้ใช้สลับจาก WiFi ไปใช้ 4G ระบบสามารถ Reconnect สัญญาณเสียงให้ใหม่ได้โดยอัตโนมัติภายในเวลาเฉลี่ย 10 วินาที

บทที่ 10: แผนพัฒนาต่อไป (Future Roadmap)

10.1 ภาพรวมการพัฒนาในระยะถัดไป

โครงการ VivaClub ระยะที่ 1 (MVP) ได้บรรลุเป้าหมายพื้นฐานในการสร้างชุมชนและระบบปรึกษาแพทย์เบื้องต้นเรียบร้อยแล้ว แผนพัฒนาในระยะถัดไปจะมุ่งเน้นไปที่การสร้างที่ยั่งยืนของแพลตฟอร์ม (Sustainability) และการนำเทคโนโลยีขั้นสูงมาเพิ่มประสิทธิภาพการรักษา

10.2 แผนงานระยะที่ 2: การสร้างที่ยั่งยืนและวิชาชีพ (V2.0)

ในระยะนี้จะมุ่งเน้นการสร้างโครงสร้างพื้นฐานทางการเงินและการยกระดับมาตรฐานทางการแพทย์:

- **ระบบเศรษฐกิจภายใน (Viva Coins & Wallet):** พัฒนาระบบกระเป๋าเงินอิเล็กทรอนิกส์เพื่อรองรับการชำระเงินค่าบริการปรึกษาแพทย์ โดยจะมีการเชื่อมต่อกับ Gateway ชื่อนำอย่าง Omise หรือ Stripe
- **การตรวจสอบตัวตนแพทย์อัตโนมัติ (OCR Verification):** นำระบบ AI มาช่วยตรวจสอบใบอนุญาตประกอบวิชาชีพเวชกรรมผ่านการสแกนเอกสาร เพื่อลดขั้นตอนการทำงานของเจ้าหน้าที่และเพิ่มความรวดเร็วในการรับสมัครแพทย์
- **ระบบใบสั่งยาอิเล็กทรอนิกส์ (E-Prescription):** พัฒนาระบบบันทึกการสั่งยาที่สอดคล้องกับกฎหมาย พ.ร.บ. เพื่อให้การรักษาจบได้ภายในแพลตฟอร์มเดียว

10.3 แผนงานระยะที่ 2: การประยุกต์ใช้ปัญญาประดิษฐ์ (AI Integration)

การนำ AI มาเป็นผู้ช่วยเบื้องต้นเพื่อคัดกรองคนไข้ได้อย่างแม่นยำและทันท่วงที:

- **Conversational PHQ-9:** เปลี่ยนจากการตอบแบบสอบถาม 9 ข้อที่น่าเบื่อเป็นการพูดคุยกับ AI Chatbot ที่มีความเห็นอกเห็นใจ (Empathy) เพื่อประเมินคะแนนสุขภาพจิตอย่างเป็นธรรมชาติ
- **Crisis Detection:** ระบบตรวจจับคำสำคัญหรือน้ำเสียงที่มีความเสี่ยงต่อการทำร้ายตนเองในระหว่างการใช้ชุมชน เพื่อให้ทีมงานหรือแพทย์สามารถเข้าช่วยเหลือได้ก่อนเกิดเหตุการณ์วิกฤต
- **AI Triage Assistant:** ระบบช่วยสรุปประวัติคนไข้เบื้องต้นให้คุณหมอก่อนเริ่มการรักษา เพื่อลดเวลาในการซักประวัติและเพิ่มประสิทธิภาพการรักษา

10.4 แผนงานระยะที่ 3: การขยายตัวสู่ระดับองค์กร (Scale & B2B)

มุ่งเน้นการขยายระบบเพื่อรองรับผู้ใช้จำนวนมากและการให้บริการในระดับองค์กร:

- **VivaCare B2B:** บริการแพ็คเกจดูแลสุขภาพจิตพนักงานสำหรับบริษัทเอกชน โดยเน้นความเป็นส่วนตัวของพนักงานและสรุปภาพรวมสุขภาพจิตขององค์กรให้ผู้บริหาร
- **Infrastructure Scaling:** การขยายเซิร์ฟเวอร์คลัสเตอร์ (LiveKit Cluster) เพื่อรองรับผู้ใช้งานพร้อมกันมากกว่า 10,000 คน และการทำฐานข้อมูลสำรอง (PostgreSQL Read Replicas) เพื่อความรวดเร็วในการเข้าถึงข้อมูล

10.5 ตารางลำดับความสำคัญ (Priority Matrix Summary)

ฟีเจอร์หลัก	ความสำคัญ	ความซับซ้อน	ระยะเวลาพัฒนา
ระบบกระเป๋าเงินและชำระเงิน	สูงมาก	สูง	4-6 สัปดาห์
ระบบแชร์หน้าจอ (Screen Sharing)	กลาง	ต่ำ	1 สัปดาห์
AI ประเมินสุขภาพจิต (Chatbot)	กลาง	สูง	4-5 สัปดาห์
ระบบตรวจจับภาวะวิกฤต (Crisis Detection)	สูงมาก	สูง	5-6 สัปดาห์
ระบบดูแลสุขภาพจิตพนักงาน (B2B)	สูง	สูงมาก	10-12 สัปดาห์

บทที่ 11: เอกสารอ้างอิงและภาคผนวก

11.1 เอกสารอ้างอิง (References)

ด้านคลินิกและสุขภาพจิต

1. **Kroenke K, Spitzer RL, Williams JBW.** (2001). The PHQ-9: Validity of a Brief Depression Severity Measure. *Journal of General Internal Medicine*, 16(9), 606–613. — Standard PHQ-9 scoring และ cutoff scores ที่ VivaClub ใช้
2. **กรมสุขภาพจิต กระทรวงสาธารณสุข.** (2566). รายงานสถานการณ์สุขภาพจิตในประเทศไทย. กรุงเทพฯ: กรมสุขภาพจิต.
3. **World Health Organization.** (2022). World Mental Health Report: Transforming Mental Health for All. Geneva: WHO.
4. พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 (PDPA). ราชกิจจานุเบกษา เล่ม 136 ตอนที่ 69 ก.

ด้านเทคโนโลยี

5. **LiveKit Inc.** (2024). LiveKit Documentation — Open Source WebRTC Infrastructure. <https://docs.livekit.io/>
6. **Django Software Foundation.** (2024). Django REST Framework Documentation. <https://www.django-rest-framework.org/>

7. **Django Software Foundation.** (2024). Django Channels Documentation. <https://channels.readthedocs.io/>
8. **Google LLC.** (2024). Flutter Documentation. <https://docs.flutter.dev/>
9. **Felix Angelov.** (2024). flutter_bloc Package Documentation. <https://bloclibrary.dev/>
10. **Pezant R., Angelov F.** (2024). GoRouter Navigation Documentation. https://pub.dev/packages/go_router
11. **Internet Engineering Task Force.** (2011). RFC 6455 — The WebSocket Protocol. IETF.
12. **Internet Engineering Task Force.** (2021). RFC 8866 — Session Description Protocol (SDP) — underlying WebRTC signaling.
13. **Caddy Community.** (2024). Caddy Documentation. <https://caddyserver.com/docs/>
14. **Firebase by Google.** (2024). Firebase Cloud Messaging Documentation. <https://firebase.google.com/docs/cloud-messaging>
15. **Docker Inc.** (2024). Docker Compose Documentation. <https://docs.docker.com/compose/>
16. **PostgreSQL Global Development Group.** (2024). PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/15/>
17. **Redis Ltd.** (2024). Redis 7 Documentation. <https://redis.io/docs/>

Appendix A: Ghost Name Generation

ghost_names.py ใช้ combination ของ:

- **24 Adjectives:** Happy, Gentle, Brave, Calm, Kind, Wise, Bright, Quiet, Warm, Soft, Bold, Swift, Clear, Deep, Vast, Free, Pure, Safe, True, Wild, Lush, Fair, Cool, Keen
- **150+ Animals:** Panda, Fox, Otter, Deer, Whale, Rabbit, Bear, Cat, Dog, Wolf, Eagle, Owl, Dove, Swan, Tiger, Lion, Frog, Duck, Crane, Moth, Bee, Ant, Elk, Boar, Lynx, Ibis, Mole, Wren, Carp, Kite, ...
- Number suffix: 1–999

ตัวอย่าง output:

- "Happy Panda #42"
- "Gentle Fox #158"
- "Brave Otter #7"
- "Calm Whale #234"

Emoji mapping (100 animals → emoji):

Total combinations: $24 \times 150 \times 999 = 3,596,400$ unique ghost names

Appendix B: Environment Variables Reference

ตารางนี้แสดง environment variables ทั้งหมดที่ backend ใช้ (ไม่รวม secret values จริง)

Variable	Example Value	Used By	วัตถุประสงค์
SECRET_KEY	django-secret-abc...	Django	Django cryptographic signing
DEBUG	FALSE	Django	Production mode
ALLOWED_HOSTS	vivaclubs.site,www.vivaclubs.site	Django	Accepted request hosts
DATABASE_URL	postgresql://user:pass@db:5432/vivaclub	Django	PostgreSQL connection
REDIS_URL	redis://:password@redis:6379/0	Django Channels	Redis connection (URL format!)
LIVEKIT_API_URL	wss://livekit.vivaclubs.site	Django	LiveKit server endpoint
LIVEKIT_API_KEY	APIxxxxxxxxxxxx	Django	LiveKit authentication
LIVEKIT_API_SECRET	secret_xxxxxxxxxxxx	Django	LiveKit JWT signing
FIREBASE_CREDENTIALS_PATH	/secrets/firebase.json	Django	FCM push notifications
SMTP_HOST	smtp.gmail.com	Django	Email server
SMTP_PORT	587	Django	Email server port
SMTP_USER	noreply@vivaclubs.site	Django	Email sender
SMTP_PASSWORD	app-password-here	Django	Gmail App Password
CLOUDINARY_CLOUD_NAME	vivaclub	Django	Media storage

CLOUDINARY_API_KEY	123456789	Django	Cloudinary authentication
CLOUDINARY_API_SECRET	secret_here	Django	Cloudinary signing
POSTGRES_DB	vivaclub_db	PostgreSQL	Database name
POSTGRES_USER	vivaclub	PostgreSQL	Database user
POSTGRES_PASSWORD	secure_password	PostgreSQL	Database password
REDIS_PASSWORD	redis_password	Redis	Redis authentication

Appendix C: PHQ-9 Scoring Table

PHQ-9 (Patient Health Questionnaire-9) เป็น validated clinical tool สำหรับ screening depression VivaClub ใช้ตามมาตรฐาน Kroenke et al. (2001):
คำถาม PHQ-9 (ภาษาไทย)

ในช่วง 2 สัปดาห์ที่ผ่านมา คุณมีปัญหาจากสิ่งต่อไปนี้บ่อยแค่ไหน?

ข้อ	คำถาม
Q1	ไม่มีความสนใจหรือความสุขในการทำสิ่งต่างๆ
Q2	รู้สึกหดหู่ใจ สิ้นหวัง หรือท้อแท้
Q3	นอนหลับยาก นอนหลับไม่สนิท หรือนอนมากเกินไป
Q4	รู้สึกเหนื่อยล้าหรือมีแรงน้อย
Q5	เบื่ออาหารหรือกินมากเกินไป
Q6	รู้สึกว่าตัวเองแย่ เป็นภาระ หรือล้มเหลว
Q7	มีปัญหาในการมีสมาธิ เช่น การอ่านหนังสือหรือดูโทรทัศน์
Q8	เคลื่อนไหวหรือพูดช้าลงจนคนอื่นสังเกตได้ หรือกระสับกระส่ายมากจนนั่งนิ่งไม่ได้
Q9	มีความคิดอยากทำร้ายตัวเองหรือคิดว่าตายเสียจะดีกว่า

ระดับคะแนน

ค่า	ความหมาย
0	ไม่เลย
1	หลายวัน (น้อยกว่าครึ่งของเวลา)
2	มากกว่าครึ่งของวัน
3	แทบทุกวัน

การแปลผล

คะแนน	ระดับ	Risk Level ใน	การดำเนินการ
0-4	Minimal	LOW	Clubhouse community
5-9	Mild depression	LOW	Self-care resources +
10-14	Moderate	MODERATE	Doctor consultation
15-19	Moderately	MODERATE	Doctor consultation
20-27	Severe	SEVERE	SOS button unlocked

ข้อควรทราบ: PHQ-9 เป็น screening tool ไม่ใช่ diagnostic tool การวินิจฉัยโรคต้องทำโดยผู้เชี่ยวชาญทางคลินิกเท่านั้น